

Product Requirements Document

Intelligent ERP Business Data Analyst Agent

Version: Draft v1

Date: 2026-05-20

Generated from PRD.md

Confidential - Internal Use

Table of Contents

- Product: Intelligent ERP Business Data Analyst Agent
- Version: Draft v1
- Date: 2026-05-20
- 1) Problem Statement
- 2) Product Vision
- 3) Goals and Non-Goals
- 4) Current State (As-Is)
- 5) Target Users
- 6) User Stories
- 7) Functional Requirements
- 8) Non-Functional Requirements
- 9) Success Metrics (KPIs)

- 10) Proposed Solution Architecture (To-Be)
- 11) Release Plan
- 12) Risks and Mitigations
- 13) Open Questions (for Product and Engineering Alignment)
- 14) Definition of Done (v1)

Product: Intelligent ERP Business Data Analyst Agent

Version: Draft v1

Date: 2026-05-20

1) Problem Statement

Current system behaves like a basic SQL bot:

- same input can produce different answers (non-deterministic/hallucinated outputs),
- complex SQL generation is often incorrect,
- business reasoning and insights are weak or generic,
- recovery from SQL/query failures is limited.

Target product should act as a reliable business data analyst, not just a query generator:

- understand business intent deeply,
- produce safe and correct complex SQL,
- run multi-step analysis,
- return trustworthy insights with evidence and confidence.

2) Product Vision

Build an AI analyst that converts natural-language business questions into:

- 1) valid, optimized SQL execution plans,
 - 2) reliable analytical computations,
 - 3) explainable business insights with traceable evidence,
- while maintaining consistency, safety, and enterprise-grade observability.

3) Goals and Non-Goals

Goals

- Deterministic and reproducible outputs for equivalent inputs.
- High SQL correctness for simple-to-complex analytical queries.
- Strong support for joins, subqueries, CTEs, window functions, period comparisons, cohorts, and KPI decomposition.
- Insight generation grounded in query results (not fabricated).

- Graceful handling of ambiguity, missing schema context, and query failure loops.
- Measurable improvement via evaluation benchmarks and runtime telemetry.

Non-Goals (v1)

- Full BI dashboard replacement.
- Autonomous schema migration / DDL writes.
- Cross-database federated querying in first release.
- Fully self-healing semantic layer without human review.

4) Current State (As-Is)

Current pipeline:

- FastAPI endpoint (/api/v1/analyze) invokes LangGraph workflow.
- Nodes: planner -> SQL generator -> DB executor -> Python analyzer -> insight synthesizer.
- LLM-driven generation (Groq model) with retry loop on SQL errors.
- Limited validation and weak determinism controls.
- Errors can propagate as 500s; grounding and confidence reporting are minimal.

5) Target Users

- Business analysts
- Operations managers
- Finance/revenue stakeholders
- Product and leadership teams needing SQL-backed insights without writing SQL manually

6) User Stories

- As a business manager, I want accurate KPI trend analysis over time so I can make planning decisions.
- As an analyst, I want complex SQL (CTEs, windows, segmentation) generated correctly so I do not manually rewrite queries.
- As an executive, I want concise insight summaries with supporting numbers and assumptions so I can trust conclusions.
- As a platform owner, I want deterministic behavior and audit logs so outputs are reproducible and debuggable.

7) Functional Requirements

FR-1 Intent Understanding and Query Decomposition

- Parse user query into:
 - business objective,
 - metrics,
 - dimensions,
 - time range,
 - filters,

- comparison logic.
- Decompose complex asks into multi-step executable plans.

FR-2 Schema-Aware Semantic Layer

- Introduce a governed schema dictionary:
 - business terms <-> table/column mappings,
 - metric definitions and formulas,
 - join paths and constraints,
 - allowed query templates by domain.
- Prevent hallucinated tables/columns via strict validation.

FR-3 Deterministic Planning and SQL Generation

- Deterministic mode defaults:
 - low/zero temperature,
 - fixed prompt/version IDs,
 - constrained output schema.
- Use structured output format for plan/SQL metadata.

FR-4 SQL Guardrails and Validation

- Pre-execution SQL checks:
 - syntax parse check,
 - schema entity existence check,
 - policy checks (read-only, row limits, forbidden patterns),
 - complexity scoring.
- Auto-repair loop with targeted error feedback.
- Hard stop + user-facing explanation after threshold retries.

FR-5 Complex Query Capability

Support for:

- multi-table joins,
- CTE chains,
- nested subqueries,
- window functions,
- YoY/MoM/WoW trends,
- percentile/ranking,
- cohort and retention analysis,
- top-N contribution analysis,
- anomaly and variance breakdown.

FR-6 Execution Orchestration

- Multi-step query execution where needed (intermediate datasets).
- Optional Python computation layer for advanced stats not practical in SQL.

- Consistent typed state contract across nodes.

FR-7 Insight Synthesis with Evidence

- Generate insights only from returned data.
- Output must include:
 - key findings,
 - metric values,
 - assumptions/filters used,
 - confidence score,
 - caveats.
- Separate facts from interpretations.

FR-8 Error Handling and Recovery UX

- Distinguish user errors, schema mismatch, LLM generation failure, DB runtime issues.
- Never silent-fail; always return actionable error context.

FR-9 Observability and Governance

- End-to-end trace ID per request.
- Node-level logs (prompt version, query hash, retries, latency).
- Redaction of secrets and PII.
- Auditability of generated SQL and final answer.

8) Non-Functional Requirements

- Reliability: >99% successful completion for supported query classes.
- Determinism: identical outputs (or near-identical within tolerance) for repeated identical inputs under deterministic mode.
- Latency: p95 response under target SLA (define by workload tier).
- Security: read-only DB role; prevent injection and unrestricted access.
- Scalability: concurrent request handling with bounded retries/timeouts.
- Maintainability: modular prompts, versioned evaluators, typed contracts.

9) Success Metrics (KPIs)

Accuracy and Quality

- SQL validity rate (first-pass and final-pass).
- Business answer correctness score (human/evaluator rated).
- Hallucination rate (non-existent schema references, fabricated claims).
- Complex-query success rate by query class.

Reliability

- Deterministic consistency score for repeated prompts.
- Retry success conversion rate.

- 500 error rate / failure taxonomy trend.

User Value

- Insight usefulness rating (thumbs-up or rubric score).
- Time-to-answer vs manual analyst baseline.
- Adoption and repeat usage per persona.

10) Proposed Solution Architecture (To-Be)

1. Intent Parser + Business DSL

- Convert user question into structured intent JSON.

2. Semantic Mapper

- Resolve business terms to governed schema.

3. Planner

- Build step-wise analytical plan (possibly multi-query).

4. SQL Composer

- Generate constrained SQL per step.

5. SQL Validator

- Static checks + schema checks + policy checks.

6. Executor

- Run SQL safely with retries and timeout strategy.

7. Analytical Engine

- Optional Python/pandas for advanced metrics.

8. Insight Engine

- Evidence-grounded narrative + confidence + caveats.

9. Evaluator Loop

- Continuous offline/online benchmarking and prompt/model tuning.

11) Release Plan

Phase 1: Stabilization (Foundational Reliability)

- Deterministic mode + structured outputs.
- Strong SQL/schema validation.
- Improved error taxonomy and response contracts.
- Baseline observability dashboards.

Phase 2: Complex Analytics Enablement

- Query decomposition + multi-step orchestration.
- Advanced SQL templates and retrieval strategy.
- Stronger retry-and-repair policy.

Phase 3: Business Intelligence Quality

- Evidence-linked insight synthesis.
- Confidence scoring and explanation quality improvements.
- Human feedback loop and eval-driven tuning.

12) Risks and Mitigations

- Risk: Model still hallucinates schema entities
- Mitigation: hard schema validator + allowlist templates + semantic dictionary.
- Risk: Complex SQL latency/cost increases
- Mitigation: query complexity budget, caching, staged execution.
- Risk: Silent analytical errors in insight text
- Mitigation: fact extraction from result tables, contradiction checks, confidence gates.
- Risk: Fragility across schema changes
- Mitigation: schema versioning + automated metadata refresh jobs.

13) Open Questions (for Product and Engineering Alignment)

- Primary vertical first (sales, inventory, finance)?
- SLA expectations by query complexity?
- Confidence thresholds for auto-answer vs ask-for-clarification?
- Should responses include full SQL by default or behind debug toggle?

14) Definition of Done (v1)

- Deterministic mode enabled by default.
- Hallucination rate reduced to agreed threshold.
- Complex SQL benchmark passes agreed success target.
- Insight outputs include evidence + confidence + caveats.
- Observability and audit logs available for all production requests.
- 500-level unhandled errors significantly reduced with classified failure outputs.